



why does OrderService get to decide what EmailService is?

Why my classes stopped building their own dependencies

from `private EmailService emailService = new EmailService();` to a container that hands my class the object it asked for

PICKS UP FROM

Entry 03 – Why JARs Exist, and What Maven Fixes

FOCUS

Dependency • Tight Coupling • DI • IoC • Beans

MENTAL MODEL LEARNED

"Don't create your dependency. Receive it."

before anything else, what even is a dependency

One class needing another — nothing more than that

```
class EmailService {
    public void sendEmail() {
        System.out.println("Email sent");
    }
}

class OrderService {
    private EmailService emailService = new EmailService();

    public void placeOrder() {
        System.out.println("Order placed");
        emailService.sendEmail();
    }
}
```

THE DEFINITION

OrderService needs **EmailService** to finish its job — that's all a dependency is. Nothing wrong with it yet. The line worth staring at is `new EmailService()` — **OrderService** is building its own dependency.

creating it yourself is the actual problem

Tight coupling — hard to change by design

TIGHTLY COUPLED THINKING

"I can travel only by this one car, with this one driver, from this one company." If that car's unavailable — I'm stuck.

LOOSELY COUPLED THINKING

"I need transportation. Car, bus, train, cab — I only care about the service, not the specific option."

TRANSLATED TO CODE

OrderService directly creates EmailService. Swap the notification channel to SMS, and I have to go back and edit OrderService itself — a change in notification logic forces a change in order logic. That's the tell.

closer, but the same mistake is still hiding in there

The object creation is still concrete

```
public interface NotificationService {
    void sendNotification();
}

public class OrderService {
    private NotificationService notificationService
        = new EmailService(); // still deciding the implementation

    public void placeOrder() {
        notificationService.sendNotification();
    }
}
```

THE CATCH

`new EmailService()` is still sitting inside `OrderService` — the variable type changed, but `OrderService` is still the one deciding which implementation to use.

*"Creating objects isn't the problem.
Creating them in the wrong place is."*

the actual fix, in one constructor

Dependency Injection — stop building, start receiving

```
public class OrderService {  
    private NotificationService notificationService;  
  
    public OrderService(NotificationService notificationService) {  
        this.notificationService = notificationService;  
    }  
}  
  
// wiring now happens outside, e.g. in Main  
NotificationService notificationService = new EmailService();  
OrderService orderService = new OrderService(notificationService);
```

WHAT ORDERSERVICE IS REALLY SAYING NOW

"Give me something that can send a notification."

constructor, setter, or field — pick on purpose

Picking the right injection style

CONSTRUCTOR INJECTION

Passed in through the constructor. Makes required dependencies explicit — the object can never exist half-built. **My default choice.**

SETTER INJECTION

Passed in through a setter method. Fits dependencies that are optional, or that might get swapped later.

FIELD INJECTION

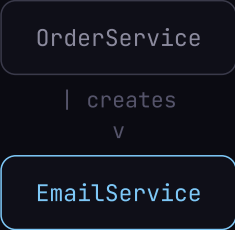
Injected straight into the field — Spring's **@Autowired**. Convenient, worse for testing. More once Spring's actually in the picture.

do not create your dependency — receive your dependency.

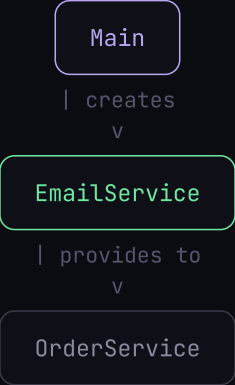
who's actually in charge of creating the object

Inversion of Control, drawn out

BEFORE – control lives inside the class



AFTER – control moves outside the class



THE REVERSAL

Earlier: the class created what it needed. Now: the class receives what it needs. That flip is called **Inversion of Control** – the normal flow is reversed.



two terms I kept mixing up until this clicked

Not the same thing – but tightly related

IoC	Principle – control of object creation moves out of the class
DI	Technique – the specific way we achieve that principle

IN ONE SENTENCE

IoC is the idea. Dependency Injection is one way to implement that idea – when I hand dependencies in from outside instead of creating them inside the class, I'm using DI to achieve IoC.



the "who wires 200 classes" moment

Spring IoC Container, and what a Bean is

THE PROBLEM AT SCALE

In plain Java, **Main** does all the creating and wiring. Fine for two classes — unworkable for the hundreds a real app has. Manually wiring all of it turns into a mess fast.

SPRING SAYS

"Don't wire every object by hand in Main. Tell me what depends on what — I'll create it, manage it, and connect it for you."

objects → plain Java code

Spring IoC Container → beans

every bean is an object — not every object is a bean

entry 04, wrapped

DI is how I stop hardcoding decisions that don't belong to the class

- ✓ a dependency is just something a class needs
- ✓ creating it yourself = tight coupling
- ✓ an interface alone doesn't remove the coupling
- ✓ DI = receive the dependency, don't build it
- ✓ constructor injection is the preferred default
- ✓ IoC = control of creation moves outside the class
- ✓ DI is the technique, IoC is the principle
- ✓ Spring's IoC Container creates, manages, injects, connects

Dependency is normal. Tight dependency is the problem – DI moves the decision outside the class, and that's exactly what IoC means in practice.

next up: Servlets · Tomcat · Spring Boot fundamentals · Component scanning · Autowiring